00 - Working in a notebook

This notebook is a tour of the tool itself, before we write any Python worth the name. It covers:

- Cells
- Code cells
- Markdown cells
- Execution order

Everything here applies to every other notebook in the course, so it is worth ten minutes now.

Cells

You are reading a **Jupyter notebook**, opened in **Positron**. Notebooks are the format we use for almost all lectures in this course, because they let us mix explanations and runnable code in a single document.

A notebook is made up of *cells*, and there are two kinds:

Cell type	Contains	Running it
`Code`	Python code	Executes the code and shows the output below
`Markdown`	Formatted text — like this cell	Displays the text as formatted text

We run a cell by selecting it and pressing `Shift + Enter`. Try it on the cell below.

In [ ]:
```
2 + 2
```

These are the commands worth knowing from the start:

Action	How
Run a cell and move to the next	`Shift + Enter`
Run a cell and stay where you are	`Ctrl + Enter` (`Cmd + Enter` on Mac)
Add a cell above or below	The `+ Code` and `+ Markdown` buttons in the toolbar
Change a cell between code and markdown	`M` to markdown, `Y` to code
Edit a markdown cell	Double-click it

Loading [MathJax]/extensions/Safe.js

Action	How
Delete a cell	The trash icon in the cell toolbar (or `d` + `d`)

Code cells

A code cell holds Python, and running it executes that Python. We can use Python as a calculator straight away.

In [ ]:
```
(2 + 4) / 8
```

Comments

It is important to document your work, so that others can follow your reasoning — and so that *you* can follow it in three weeks' time. We write comments inside code cells using the `#` symbol. Everything after `#` on that line is ignored by Python.

In [ ]:
```
# This is a comment explaining the code below
2 * 3
```

Displaying output

By default, a code cell only displays the result of its **last** expression. In the cell below, the first result never appears.

In [ ]:
```
2 + 2
2 * 5
```

To display several results from the same cell, we use the `print` function.

In [ ]:
```
print(2 + 2)
print(2 * 5)
```

> 💡 **Tip:** This catches everyone out at some point. If a result you expected is missing, check whether you wrapped it in `print`.

Markdown cells

Markdown cells are what make a notebook a document rather than a script. They use a simple formatting language called Markdown: you type plain text with a few special characters, and it is displayed as formatted text.

The sections below show the Markdown code first, then what it produces.

Loading [MathJax]/extensions/Safe.js

Headings

One to four `#` symbols at the start of a line give headings of decreasing size:

```
# Heading level 1
## Heading level 2
### Heading level 3
```

Which produces:

Heading level 1

Heading level 2

Heading level 3

Emphasis

Asterisks around text make it bold or italic:

```
*italic text* and **bold text**
```

Which produces:

italic text and **bold text**

Lists

A `-` at the start of a line gives a bullet point, and a number followed by a `.` gives a numbered list:

```
- first item
- second item
- third item

1. first step
2. second step
3. third step
```

Which produces:

- first item
- second item

Loading [MathJax]/extensions/Safe.js

1. first step
2. second step
3. third step

Code inside text

Backticks mark text as code, which is how we refer to Python inside an explanation:

```
The `print` function displays a value.
```

Which produces:

The `print` function displays a value.

Links, tables and mathematics

```
[The Python documentation](https://docs.python.org)

| Course | Credits |
|---|---|
| BAN405 | 7.5 |

The formula $C = (F - 32) \times 5/9$ converts Fahrenheit to
Celsius.
```

Which produces:

The Python documentation

Course	Credits
BAN405	7.5

The formula $C = (F - 32) \times 5/9$ converts Fahrenheit to Celsius.

Execution order

This is the one thing about notebooks that genuinely catches people out, so it is worth being precise about it.

A notebook does not read the cells in the order they appear on your screen. It remembers what you have run, in the **order you ran it**, and everything you create stays in memory until you restart.

The number in square brackets to the left of a code cell — `[1]`, `[2]`, `[3]` — tells you the order in which cells were actually executed.

Loading [MathJax]/extensions/Safe.js

In [ ]:
```
message = "first value"
```

In [ ]:
```
print(message)
```

Now imagine you go back, change the cell above to `message = "second value"`, and run only that cell. The `print` cell still shows the old output on screen, even though it is now wrong. Nothing in the notebook warns you about this.

The same problem appears in reverse: a notebook that works perfectly for you may fail for someone else, because it depends on a cell you ran once and later deleted.

> 💡 **Tip:** Before you trust a notebook — and always before you hand one in — use **Restart** and then **Run All**. If the notebook runs cleanly from top to bottom, the results you see are the results your code actually produces.

> ⚠️ **Warning:** Restarting also erases every variable you have created. That is the point of it, but it does mean you have to run your cells again afterwards.

Additional resources

- Positron user guide — the official documentation for Jupyter notebooks in Positron
- Markdown guide: basic syntax — everything Markdown can do, with examples
- Jupyter notebook documentation — the underlying format, if you are curious about how it works

Loading [MathJax]/extensions/Safe.js